

Guía de Aprendizaje: Control de Flujo en C

Monitorías de Sistemas Operativos

1. Motivación

Los programas no siempre se ejecutan en línea recta. Necesitamos que la CPU tome decisiones o repita tareas condicionalmente (como un sistema operativo que espera una señal de teclado o gestiona múltiples procesos en un bucle).

2. Idea Fundamental

El control de flujo permite alterar el orden secuencial de ejecución de las instrucciones mediante **condicionales** y **bucles**.

3. Sintaxis Básica

```
if(condicion){ ... } else { ... }  
for(inicializacion; condicion; incremento){ ... }  
while(condicion){ ... }
```

4. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>  
2  
3 int main(){  
4     int n = 3;  
5  
6     if(n > 0){  
7         for(int i = 0; i < n; i++){  
8             printf("Proceso %d ejecutándose\n", i);  
9         }  
10    }  
11  
12    return 0;  
13 }
```

5. Explicación Paso a Paso

1. Se evalúa si `n` es mayor a 0.
2. Si es verdadero, entra al bucle `for`.

3. El bucle se repite mientras $i < n$.
4. En cada paso se imprime un mensaje y se incrementa i .

6. Qué ocurre en memoria

A nivel de bajo nivel, esto se traduce en instrucciones de salto (**jump**) en ensamblador. Las variables de control (como i) viven en el Stack de la función **main**.

7. Patrones comunes de uso

El bucle infinito para monitorización o servicios:

```
1 while(1){
2     // Escuchar peticiones
3     if(recibir_senal()) break;
4 }
```

8. Errores Comunes

- **Bucle Infinito:** La condición de salida nunca se vuelve falsa.
- **Error de Off-by-one:** El bucle se ejecuta una vez más o una vez menos de lo deseado ($i \leq n$ vs $i < n$).

9. Buenas Prácticas

- Indentar correctamente el código para visualizar la jerarquía.
- Evitar el uso de **goto** (hacerlo sólo si es para manejo de errores en C crítico).

10. Ejercicios

1. Escriba un programa que imprima los números pares del 1 al 100 usando un bucle **for**.
2. Implemente un programa que lea un número e indique si es primo.